

Logistic Regression

Classification, Maximum Likelihood, and the
Loss-Minimization Template

Tu Li

Fall 2026

Econ 453 University of Arizona

Roadmap

1. From Continuous to Binary Responses
2. Why OLS Fails for Binary Y
3. The Logistic Regression Model
4. Estimation — Maximum Likelihood
5. The Classification Rule
6. Multiple Predictors
7. Non-Linear Logistic Regression
8. Regularized Logistic Regression
9. Summary

1. From Continuous to Binary Responses

1.1 What We Have Done So Far

Lectures 6 through 11 estimated a regression function

$$Y = f(X) + \epsilon,$$

where Y is a **real number** — wage, sales, price, log-return.

Estimators we built:

- **OLS** on a linear model (L6, L9).
- **Ridge / LASSO**, OLS plus a penalty (L10).
- **Polynomial / basis regression**, OLS on transformed features (L11).

All shared the same loss-minimization template: pick a model family, pick a loss (RSS), minimise.

This lecture changes the response type. What if Y takes only two values?

1.2 Binary Responses in Practice

Many response variables in business and science are **binary** — the outcome is one of two categories. Examples:

- **Loan default.** $Y = 1$ if the borrower defaults, $Y = 0$ otherwise.
- **Email spam.** $Y = 1$ if the email is spam, $Y = 0$ otherwise.
- **Disease diagnosis.** $Y = 1$ if the patient has the disease, $Y = 0$ otherwise.
- **Click-through.** $Y = 1$ if the user clicks the ad, $Y = 0$ otherwise.

Notation reminder. We code the two categories as $Y \in \{0, 1\}$, with 1 usually denoting “the event” (default, spam, disease, click) and 0 its absence. The choice is a convention — nothing in the math forces it.

1.3 The Naïve Idea — Just Run OLS

Given training data $(x_1, y_1), \dots, (x_n, y_n)$ with each $y_i \in \{0, 1\}$, the simplest thing we could do is pretend Y is continuous and run OLS:

$$\hat{y}_{\text{OLS}} = \hat{\beta}_0 + \hat{\beta}_1 x.$$

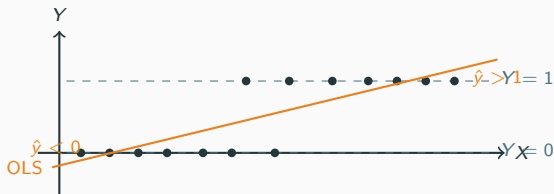
No new machinery — the L6 formulas apply unchanged. So: does it work?

Section 2 says **no, and here is why**.

2. Why OLS Fails for Binary Y

2.1 Predictions Escape $[0, 1]$

Plot the data and the OLS line:



Two observations:

- For small x , the line predicts $\hat{y} < 0$ — negative.
- For large x , the line predicts $\hat{y} > 1$ — bigger than 1.

If we want $\hat{y}$ to mean “the probability that $Y = 1$ given $X = x$,” **neither value is meaningful**. A probability lives in $[0, 1]$. The OLS line refuses to.

2.2 What Does $\hat{y}$ Even Mean?

Even setting the range aside: OLS produces a continuous $\hat{y} \in \mathbb{R}$, but we eventually want a **class label** — 0 or 1.

Two related questions OLS cannot directly answer:

- **What is the probability that $Y = 1$?** That is the object that has the most uses — we want to be able to say “this borrower has a 12% chance of default.”
- **How confident is the prediction?** A probability near 0.5 means the model is unsure; near 0.99 means it is certain. OLS does not deliver this calibration.

We need a method whose output is, by construction, a probability in $[0, 1]$.

2.3 What We Actually Want

The right target

Model the conditional probability

$$p(x) \equiv \mathbb{P}(Y = 1 \mid X = x)$$

directly, with a function that returns values in $[0, 1]$ for every x .

Once we have $p(x)$:

- We can report the probability itself — “12% chance of default.”
- We can convert it to a class label by a threshold — “predict default if $p(x) \geq 0.5$.”
- Inference, regularization, and non-linear extensions follow the same template as for $\hat{f}$ in L6–L11.

Next: the specific functional form.

3. The Logistic Regression Model

3.1 Two Strategies, Side by Side

The conceptual spine of this lecture is a comparison between OLS and logistic regression. Both are *parametric* models with a linear combination $\beta_0 + \beta_1 X$ inside them. The difference is what we put around that linear combination.

	OLS (L6)	Logistic regression
What we model	$\mathbb{E}[Y X] = \beta_0 + \beta_1 X$	$\mathbb{P}(Y=1 X) = \frac{e^{\beta_0 + \beta_1 X}}{1 + e^{\beta_0 + \beta_1 X}}$
Why this form	Y continuous in $\mathbb{R}$	$Y \in \{0, 1\}$; output in $[0, 1]$
Loss	RSS	$-\ell(\beta)$
Solve	closed form	computer
Output	$\hat{y}_{\text{new}} \in \mathbb{R}$	$\hat{p} \in [0, 1]$
Used for	prediction, inference	prediction, classification

Same template (find β by optimising a function of it), different ingredients. The rest of §3–§5 unpacks the right column.

3.2 The Logistic Regression Model

We saw in §2 that running OLS on a $\{0, 1\}$ response can produce predicted “probabilities” below 0 or above 1. To stay in $[0, 1]$ we model the conditional probability directly with a specific functional form.

Definition — logistic regression (one predictor)

Let X be a predictor and $Y \in \{0, 1\}$ the response. The **logistic regression model** is

$$p(x) \equiv \mathbb{P}(Y = 1 \mid X = x) = \frac{e^{\beta_0 + \beta_1 x}}{1 + e^{\beta_0 + \beta_1 x}}.$$

Same predictor structure as OLS: a linear combination $\beta_0 + \beta_1 X$ of the predictor with two unknown coefficients. The $e^{\cdot} / (1 + e^{\cdot})$ form is what keeps the output in $[0, 1]$.

3.2 The Logistic Regression Model 2

Notation reminder. We use $p(x)$ as shorthand for $\mathbb{P}(Y = 1 \mid X = x)$.

The complement is $1 - p(x) = \mathbb{P}(Y = 0 \mid X = x)$. Plugging the model in and simplifying:

$$1 - p(x) = \frac{1}{1 + e^{\beta_0 + \beta_1 x}}.$$

Sanity check:

- As $\beta_0 + \beta_1 x \rightarrow +\infty$, $p(x) \rightarrow 1$.
- As $\beta_0 + \beta_1 x \rightarrow -\infty$, $p(x) \rightarrow 0$.
- At $\beta_0 + \beta_1 x = 0$, $p(x) = \frac{1}{2}$.

Monotone in the linear part, output strictly between 0 and 1 — exactly what §2.3 demanded.

3.3 The Log-Odds (Logit) Form

Algebraic rearrangement of the model from §3.2. With

$$p(x) = \mathbb{P}(Y = 1 \mid X = x),$$

$$p(x) = \frac{e^{\beta_0 + \beta_1 x}}{1 + e^{\beta_0 + \beta_1 x}}, \quad 1 - p(x) = \frac{1}{1 + e^{\beta_0 + \beta_1 x}}.$$

Divide and simplify:

$$\frac{p(x)}{1 - p(x)} = e^{\beta_0 + \beta_1 x}.$$

The quantity $p(x)/(1 - p(x))$ is the **odds** that $Y = 1$ given $X = x$.

Take logs:

3.3 The Log-Odds (Logit) Form 2

Key Result — log-odds (logit) form

$$\log \frac{p(x)}{1 - p(x)} = \beta_0 + \beta_1 x.$$

Reading. The model is *linear in x on the log-odds scale*. The logit transform is what makes a linear-in-parameters model fit a $[0, 1]$ -bounded probability. This is the bridge between the OLS world and the classification world.

3.4 Coefficient Interpretation

From the log-odds form, compare two inputs whose x values differ by one unit:

$$\log \frac{p(x+1)}{1-p(x+1)} - \log \frac{p(x)}{1-p(x)} = \beta_1.$$

So:

- $\hat{\beta}_1$ is the fitted **difference in log-odds** when x is one unit higher.
- $e^{\hat{\beta}_1}$ is the fitted **odds ratio** for those two inputs.
- $\hat{\beta}_0$ is the log-odds when $x = 0$.

Interpretation caveat. A one-unit change in X does not produce a fixed change in $\mathbb{P}(Y = 1)$ itself — that depends on the starting probability (the model is non-linear in X). Coefficient interpretation lives most naturally on the log-odds scale. These are predictive associations, not causal effects unless the research design supports a causal interpretation.

4. Estimation — Maximum Likelihood

4.1 The Strategy Gap

OLS **minimises** the residual sum of squares:

$$\hat{\beta}_{\text{OLS}} = \arg \min_{\beta} \sum_{i=1}^n (y_i - \beta_0 - \beta_1 x_i)^2.$$

Logistic regression **maximises the likelihood** of the observed data:

$$\hat{\beta}_{\text{logit}} = \arg \max_{\beta} L(\beta).$$

Same loss-minimization template (find β by optimising a function of it), different function to optimise. Section 4.2 explains what “likelihood” means here.

4.2 The MLE Principle, Intuitively

Maximum Likelihood Estimation (intuitive)

Given a parametric model and observed data, the **maximum likelihood estimate (MLE)** is the parameter value under which the observed data are *most probable*.

In words: the MLE asks “what value of β makes the data we saw look the least surprising?” And that is the value we report.

Why it makes sense. Different choices of β give the data different probabilities. If one choice gives the observed data probability 0.01 and another gives 0.20, the second choice is twenty times more compatible with what we actually saw. The MLE picks the most compatible parameter.

Same idea you’ve used informally all year: “which model best explains this data.”

4.3 The Likelihood for One Observation

Each observation i is a Bernoulli outcome with success probability

$$p(x_i) = \frac{e^{\beta_0 + \beta_1 x_i}}{1 + e^{\beta_0 + \beta_1 x_i}}$$

(the model from §3.2 evaluated at x_i). The probability of observing the actual $y_i \in \{0, 1\}$ is

$$\mathbb{P}(Y_i = y_i \mid x_i; \beta) = p(x_i)^{y_i} (1 - p(x_i))^{1-y_i}.$$

Notation reminder. The compact formula collapses to:

- if $y_i = 1$: just $p(x_i)$.
- if $y_i = 0$: just $1 - p(x_i)$.

One expression, two cases.

Notice β enters through $p(x_i)$. Changing β changes how probable each observation is.

4.4 The Likelihood Across n Observations

Observations are independent, so the joint probability of seeing all $y_1, \dots, y_n$ is the product:

Definition — likelihood function

$$L(\beta) = \prod_{i=1}^n p(x_i)^{y_i} (1 - p(x_i))^{1-y_i}.$$

Software actually maximises the **log-likelihood**, which turns the product into a sum:

$$\ell(\beta) = \log L(\beta) = \sum_{i=1}^n \left[y_i \log p(x_i) + (1 - y_i) \log(1 - p(x_i)) \right].$$

4.4 The Likelihood Across n Observations 2

Equivalently, the MLE is

$$\hat{\beta} = \arg \min_{\beta} \{ -\ell(\beta) \}.$$

This puts logistic regression squarely in the same loss-minimization template as OLS: *minimise* a function of β . The function is $-\ell$ instead of RSS.

4.5 No Closed Form — Computer Does It

In OLS, setting partial derivatives of RSS to zero gives a clean closed-form $\hat{\beta} = (\mathbf{X}^\top \mathbf{X})^{-1} \mathbf{X}^\top \mathbf{y}$ (L6 Appendix D, L9 §2).

In logistic regression, setting partials of $\ell(\beta)$ to zero gives a system of equations that has **no closed-form solution**. The model is non-linear in β , and the equations cannot be solved by algebra.

Computer does it. Specialised iterative algorithms (Newton–Raphson, iteratively reweighted least squares, gradient descent) compute $\hat{\beta}$ numerically. The same “computer does it” convention we used for LASSO in L10 §5.6.

What we get back. A numeric vector $(\hat{\beta}_0, \hat{\beta}_1, \dots)$, plus a fitted probability $\hat{p}(x)$ for any input x .

5. The Classification Rule

5.1 From Probability to Class Label

Once we have

$$\hat{p}(x) = \frac{e^{\hat{\beta}_0 + \hat{\beta}_1 x}}{1 + e^{\hat{\beta}_0 + \hat{\beta}_1 x}},$$

predicting a class label is a one-step rule:

Key Result — classification rule (default threshold)

$$\hat{Y}_{\text{new}} = \begin{cases} 1 & \text{if } \hat{p}(x_{\text{new}}) \geq 0.5, \\ 0 & \text{if } \hat{p}(x_{\text{new}}) < 0.5. \end{cases}$$

In words: predict the more likely class. The 0.5 threshold is the default; we revisit it in §5.3.

5.2 The Decision Boundary

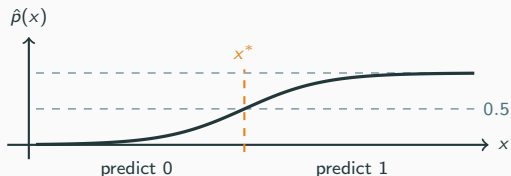
The classification rule depends only on whether $\hat{p} \geq 0.5$. Solve for the boundary:

$$\begin{aligned}\hat{p}(x) = 0.5 &\iff \frac{e^{\hat{\beta}_0 + \hat{\beta}_1 x}}{1 + e^{\hat{\beta}_0 + \hat{\beta}_1 x}} = 0.5 \\ &\iff e^{\hat{\beta}_0 + \hat{\beta}_1 x} = 1 \\ &\iff \hat{\beta}_0 + \hat{\beta}_1 x = 0 \\ &\iff x = -\hat{\beta}_0 / \hat{\beta}_1 \equiv x^*.\end{aligned}$$

Decision rule, rewritten. Compare x to a single threshold:

- If $\hat{\beta}_1 > 0$: predict 1 when $x \geq x^*$.
- If $\hat{\beta}_1 < 0$: predict 1 when $x \leq x^*$.

5.2 The Decision Boundary 2



The decision boundary in the one-predictor case is a single number. In §6 with p predictors it becomes a hyperplane; in §7 it can become curved.

5.3 When the Threshold Should Not Be 0.5

The 0.5 threshold treats false positives and false negatives as equally costly. In practice the two error types often differ:

- **Medical screening.** Missing a real disease is much worse than a false alarm $\Rightarrow$ lower the threshold to catch more positives.
- **Spam filtering.** Marking real mail as spam is worse than letting some spam through $\Rightarrow$ raise the threshold.
- **Loan approval.** Approving a defaulter is much costlier than rejecting a safe borrower $\Rightarrow$ lower the threshold for “default.”

The principle. The threshold is a separate decision from the probability model. Pick it from the application’s cost structure (or from validation data).

5.4 Worked Example — Loan Default by Income

Setup. Fabricated data: $n = 200$ borrowers; $Y = 1$ if the borrower defaulted, $X =$ annual income (\$1,000s). Fit logistic regression by software.

Software output (fabricated).

$$\hat{\beta}_0 = 6.10, \quad \hat{\beta}_1 = -0.080, \quad \hat{p}(x) = \frac{e^{6.10 - 0.080x}}{1 + e^{6.10 - 0.080x}}.$$

Predicted probabilities at three sample borrowers. Let $z = 6.10 - 0.080x$; then $\hat{p} = e^z / (1 + e^z)$:

income x (\$1k)	z	$\hat{p}(x)$	class @ 0.5
40	2.90	0.948	default (1)
75	0.10	0.525	default (1)
100	-1.90	0.130	repay (0)

5.4 Worked Example — Loan Default by Income 2

Decision boundary. Solve $6.10 - 0.080x^* = 0 \Rightarrow x^* = 76.25$.

Reading. The model predicts default for borrowers with income below \$76,250; above that threshold, it predicts repayment. The coefficient $\hat{\beta}_1 = -0.080$ says that income \$1,000 higher is associated with fitted log-odds of default lower by 0.080. Equivalently, the fitted odds at income $x + 1$ are $e^{-0.080} \approx 0.92$ times the fitted odds at income x .

6. Multiple Predictors

6.1 The Model with p Predictors

Same trick as L9: stretch the linear combination from $\beta_0 + \beta_1 X$ to $\beta_0 + \beta_1 X_1 + \cdots + \beta_p X_p$, and plug it into the same exponential form.

Definition — multiple logistic regression

For p predictors, let $\mathbf{x} = (x_1, \dots, x_p)^\top \in \mathbb{R}^p$. Include the intercept by defining $\tilde{\mathbf{x}} = (1, x_1, \dots, x_p)^\top \in \mathbb{R}^{p+1}$ and $\boldsymbol{\beta} = (\beta_0, \beta_1, \dots, \beta_p)^\top \in \mathbb{R}^{p+1}$. Then $\tilde{\mathbf{x}}^\top \boldsymbol{\beta} = \beta_0 + \beta_1 x_1 + \cdots + \beta_p x_p$, and

$$p(\mathbf{x}) \equiv \mathbb{P}(Y = 1 \mid \mathbf{x}) = \frac{e^{\tilde{\mathbf{x}}^\top \boldsymbol{\beta}}}{1 + e^{\tilde{\mathbf{x}}^\top \boldsymbol{\beta}}}.$$

Same functional form as §3.2, with the longer linear combination inside the exponentials. Nothing else changes.

6.2 Estimation by Maximum Likelihood

For observation i , augment the p predictor values with a leading 1: $\tilde{\mathbf{x}}_i = (1, x_{i1}, \dots, x_{ip})^\top$. The probability of $Y_i = 1$ is

$$p(\mathbf{x}_i) = \frac{e^{\tilde{\mathbf{x}}_i^\top \boldsymbol{\beta}}}{1 + e^{\tilde{\mathbf{x}}_i^\top \boldsymbol{\beta}}}.$$

Estimation is unchanged in principle. Same Bernoulli likelihood as §4, with the longer linear combination plugged in:

$$\ell(\boldsymbol{\beta}) = \sum_{i=1}^n [y_i \log p(\mathbf{x}_i) + (1 - y_i) \log(1 - p(\mathbf{x}_i))],$$

and the MLE maximises it. Computer does it. No closed form.

Same design-matrix convention as L9. Stack the augmented rows $\tilde{\mathbf{x}}_i^\top$ into $\mathbf{X} \in \mathbb{R}^{n \times (p+1)}$: one intercept column plus p predictor columns.

6.2 Estimation by Maximum Likelihood 2

Logistic regression has no closed-form estimator, but software uses this same data layout when maximising the likelihood.

6.3 Coefficient Interpretation

Multiple-predictor reading mirrors L9:

- $\hat{\beta}_j$ is the fitted difference in log-odds of $Y = 1$ when X_j is one unit higher, **holding the other predictors fixed**.
- $e^{\hat{\beta}_j}$ is the corresponding fitted odds ratio.
- $\hat{\beta}_0$ is the log-odds when every $X_j = 0$.

As in L9, the “holding others fixed” clause is the whole point of multiple regression: a predictor that seemed important in isolation may be insignificant once others are controlled for. This is an adjusted association, not automatically a causal effect.

6.4 Decision Boundary in Higher Dimensions

Same classification rule: predict 1 when $\hat{p} \geq 0.5$. Equivalently:

$$\hat{Y} = 1 \iff \hat{\beta}_0 + \hat{\beta}_1 x_1 + \cdots + \hat{\beta}_p x_p \geq 0.$$

The decision boundary $\{ \mathbf{x} : \hat{\beta}_0 + \hat{\beta}_1 x_1 + \cdots + \hat{\beta}_p x_p = 0 \}$ is a hyperplane in $\mathbb{R}^p$:

- $p = 1$: a single point on the real line.
- $p = 2$: a straight line in the plane.
- general p : an affine hyperplane.

Logistic regression's decision boundary is linear in the predictors. Section 7 shows how to bend it.

6.5 Worked Example — Default with Four Predictors

Setup. Extend the §5.4 dataset: predict default from $X_1 = \text{income}$ (\$1k), $X_2 = \text{credit score}$, $X_3 = \text{debt ratio}$, $X_4 = \text{age}$.

Software output (fabricated).

Predictor	$\hat{\beta}_j$	odds ratio $e^{\hat{\beta}_j}$
(intercept)	5.20	—
income (\$1k)	-0.062	0.94
credit score (pts)	-0.014	0.986
debt ratio (%)	0.075	1.078
age (years)	-0.021	0.979

Reading. Each row holds the other predictors fixed. Within the fitted model, income \$1,000 higher is associated with about 6% lower predicted odds of default; a debt ratio one percentage point higher is associated with about 8% higher predicted odds. The intercept gives the baseline

6.5 Worked Example — Default with Four Predictors 2

log-odds for a borrower with all predictors at zero (not a meaningful borrower, but a useful anchor).

Forward pointer. How precisely is each $\hat{\beta}_j$ estimated? Standard errors and CIs follow the same logic as in L7/L9, with the SE formula again computed by software — “computer does it.”

7. Non-Linear Logistic Regression

7.1 Bending the Boundary — The L11 Trick

The decision boundary of plain logistic regression is a hyperplane (§6.4). For non-linear classification problems — like the concentric-circles toy data below — a hyperplane cannot separate the classes.

Solution: **the same trick as in L11**. Replace the columns of $\mathbf{X}$ with non-linear features $\phi_j(\mathbf{x})$ — polynomial monomials, sines/cosines, splines, indicators — and fit logistic regression on the transformed design matrix.

Sections 7.2–7.3 develop this.

7.2 Polynomial Logistic Regression

Take one predictor x and feed the polynomial design matrix from L11 §5 into the same exponential form:

$$\mathbb{P}(Y = 1 \mid x) = \frac{e^{\beta_0 + \beta_1 x + \beta_2 x^2 + \dots + \beta_d x^d}}{1 + e^{\beta_0 + \beta_1 x + \beta_2 x^2 + \dots + \beta_d x^d}}.$$

The design matrix is identical to L11. It has one intercept column plus d transformed predictor columns, so here $p = d$ and the total number of columns is $p + 1 = d + 1$:

$$\mathbf{X}_{\text{poly}} = \begin{pmatrix} 1 & x_1 & x_1^2 & \cdots & x_1^d \\ \vdots & \vdots & \vdots & \ddots & \vdots \\ 1 & x_n & x_n^2 & \cdots & x_n^d \end{pmatrix}.$$

Same loss-minimization template.

7.2 Polynomial Logistic Regression 2

- Same logistic form around the linear part.
- Same likelihood, same MLE, same “computer does it” optimisation.
- Only difference: the columns of $\mathbf{X}$ are powers of one predictor.

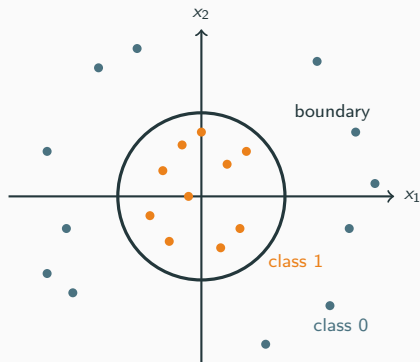
Hyperparameter d . The degree d is the same kind of hyperparameter as in L11 §3.3: not estimated by maximising the likelihood, fixed from outside, picked by cross-validation (L13).

7.3 Non-Linear Decision Boundaries

When the linear part is a polynomial in x , the decision boundary is the set of x where $\hat{\beta}_0 + \hat{\beta}_1x + \cdots + \hat{\beta}_dx^d = 0$ — a polynomial level set.

Two-predictor picture (degree-2 in x_1, x_2). The linear part $\hat{\beta}_0 + \hat{\beta}_1x_1 + \hat{\beta}_2x_2 + \hat{\beta}_3x_1^2 + \hat{\beta}_4x_1x_2 + \hat{\beta}_5x_2^2 = 0$ can be a circle, ellipse, parabola, or hyperbola:

7.3 Non-Linear Decision Boundaries 2



Plain (linear) logistic regression could not separate these classes with a straight line. Polynomial logistic regression can.

8. Regularized Logistic Regression

8.1 Adding a Penalty to the Log-Likelihood

Recall L10: ridge and LASSO add a size penalty to the OLS objective. We do the same here, replacing RSS with the negative log-likelihood:

Definition — LASSO logistic regression

For a hyperparameter $\lambda \geq 0$,

$$\hat{\beta}_{\text{LASSO-logit}} = \arg \min_{\beta} \left\{ -\ell(\beta) + \lambda \sum_{j=1}^p |\beta_j| \right\}.$$

Ridge logistic uses a squared penalty on the slope coefficients:

$$\hat{\beta}_{\text{ridge-logit}} = \arg \min_{\beta} \left\{ -\ell(\beta) + \lambda \sum_{j=1}^p \beta_j^2 \right\}.$$

In both objectives the intercept β_0 is not penalised.

8.1 Adding a Penalty to the Log-Likelihood 2

Same template, different loss. OLS got a penalty in L10; logistic gets the same penalty here. The loss-minimization template absorbs both.

8.2 Why It Works — Same Story as L10

- **Ridge logistic.** Shrinks every slope coefficient toward zero; keeps all predictors; stabilises the fit when p is large relative to n or when predictors are correlated.
- **LASSO logistic.** Same geometry as L10 §5.4 — the ℓ_1 ball has corners on the axes, so some $\hat{\beta}_j, j \geq 1$, come out *exactly zero*. Automatic variable selection; the intercept remains unpenalised.

Bias–variance reading.

- Both add a small bias to the unpenalised MLE.
- Both reduce variance.
- The reduction is bigger than the bias added for sensible λ — exactly the L10 §3 argument applied to the new loss.

8.3 Hyperparameter λ

λ is a **hyperparameter**

Just like in L10 §4.2, the number $\lambda \geq 0$ is fixed from outside the optimisation. It controls how aggressively we shrink the coefficients. Specifically:

- $\lambda = 0$ recovers the unpenalised logistic MLE.
- $\lambda \rightarrow \infty$ shrinks all slope coefficients toward zero while leaving the intercept unpenalised.

For now, treat λ as given. How to pick it from data: cross-validation, formally treated in Lecture 13 — the same procedure that picks λ for ridge and LASSO in L10 and the degree d for polynomials in L11.

Computer does it. Software fits ridge / LASSO logistic with one extra argument (the value of λ). No new optimisation theory.

8.4 When to Use Which

Three settings to remember:

Setting	Method	Why
$n \gg p$, predictors all relevant	plain logistic	no need for shrinkage
p large relative to n ; all predictors carry signal	ridge logistic	stabilise; keep all variables
p large relative to n ; only a few predictors matter	LASSO logistic	sparse and interpretable

Same chooser as L10 §6, just with “logistic” attached.

9. Summary

9.1 Take Away — Foundations

Three big ideas

- **OLS fails for binary Y .** Predictions escape $[0, 1]$ and have no probabilistic reading. We model $\mathbb{P}(Y = 1 \mid X)$ directly as $e^{\beta_0 + \beta_1 X} / (1 + e^{\beta_0 + \beta_1 X})$, guaranteeing the output stays in $[0, 1]$.
- **Same template, different loss.** The OLS loss-minimization recipe (L6, L9, L10, L11) carries over: find β by optimising a function of it. The function is the negative log-likelihood. No closed form; computer does it.
- **Classification rule.** Logistic regression outputs a probability; a threshold (default 0.5, adjustable) turns it into a class label. The decision boundary is a hyperplane in $\mathbf{x}$ — bendable via polynomial / basis features (L11), shrinkable via the L10 penalties.

9.2 Take Away — Key Formulas

The L12 toolbox

- **Logistic model:** $\mathbb{P}(Y = 1 \mid \mathbf{x}) = \frac{e^{\tilde{\mathbf{x}}^\top \boldsymbol{\beta}}}{1 + e^{\tilde{\mathbf{x}}^\top \boldsymbol{\beta}}}$, where $\tilde{\mathbf{x}} = (1, x_1, \dots, x_p)^\top$.
- **Log-odds:** $\log \frac{p(\mathbf{x})}{1-p(\mathbf{x})} = \tilde{\mathbf{x}}^\top \boldsymbol{\beta}$.
- **Likelihood:** $L(\boldsymbol{\beta}) = \prod_i p(\mathbf{x}_i)^{y_i} (1 - p(\mathbf{x}_i))^{1-y_i}$.
- **MLE:** $\hat{\boldsymbol{\beta}} = \arg \max_{\boldsymbol{\beta}} \ell(\boldsymbol{\beta})$ — no closed form; software solves.
- **Classification rule:** predict 1 iff $\hat{p}(\mathbf{x}_{\text{new}}) \geq 0.5$ (default threshold).
- **Decision boundary:** $\hat{\beta}_0 + \hat{\beta}_1 x_1 + \dots + \hat{\beta}_p x_p = 0$.
- **LASSO logistic:** replace RSS in L10 §5 with $-\ell(\boldsymbol{\beta})$.

9.3 Where Each Idea Reappears

Forward pointers

- **Lecture 13 (resampling)**. Cross-validation picks λ for ridge / LASSO logistic the same way it picks λ in L10 and the degree d in L11. One procedure, many uses.
- **Lecture 14 (random forests)**. A fundamentally non-parametric classifier — recursive partitioning rather than a global parametric model. Sidesteps the curse-of-dimensionality issues that bite polynomial-basis logistic in high dimensions.

Closing. OLS was the first parametric ML method (L6); logistic regression is the same template, retargeted to a binary response. Everything since has been variations on this loss-minimization theme.